

Продвинутое управление памятью: `ArrayPool<T>` и `MemoryPool<T>`

C# Developer. Advanced



Проверить, идет ли запись

Меня хорошо видно & слышно?





Екатерина Меттус

.NET разработчик

 @mettus_katerina

 mettus.ekaterina@gmail.com

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в чате учебной группы



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Маршрут вебинара

Пул объектов

ArrayPool<T>

MemoryPool<T>

Правила использования пулов

Рефлексия

Цели вебинара

К концу занятия вы сможете

1. Применять паттерн "пул объектов"

2. Использовать `ArrayPool<T>`

3. Использовать `MemoryPool<T>`

Смысл

Для чего вам это уметь

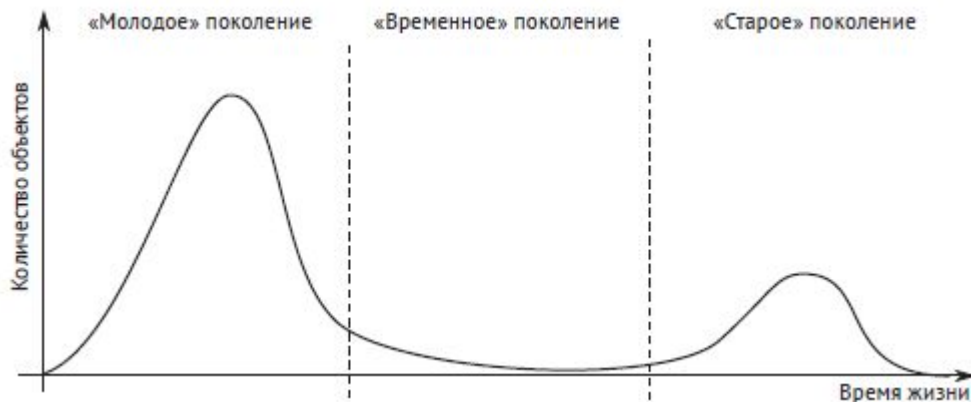
Проектировать системы, устойчивые к пиковым нагрузкам за счет пулинга буферов



Почему 3 поколения?

Самые молодые объекты живут недолго.

Чем дольше живет объект, тем более вероятно, что он проживет еще.



stackalloc vs new()

```
var buffer = new byte[1024];
```

```
ldc.i4 1024
```

```
newarr [System.Private.CoreLib]System.Byte
```

```
Span<byte> buffer = stackalloc byte[128];
```

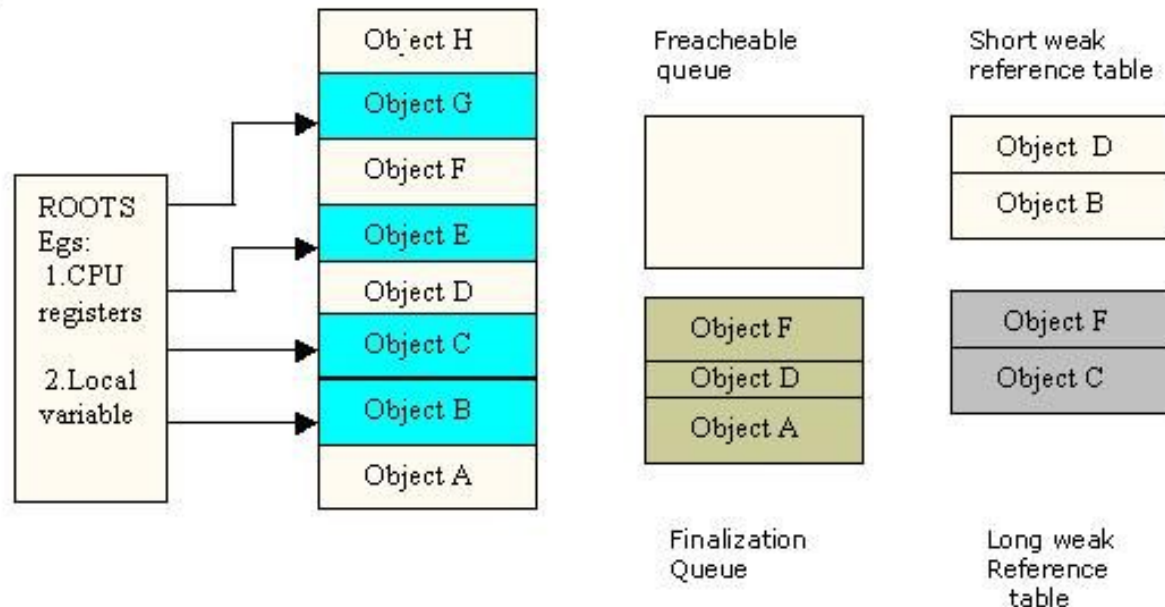
```
ldc.i4 128
```

```
localloc
```



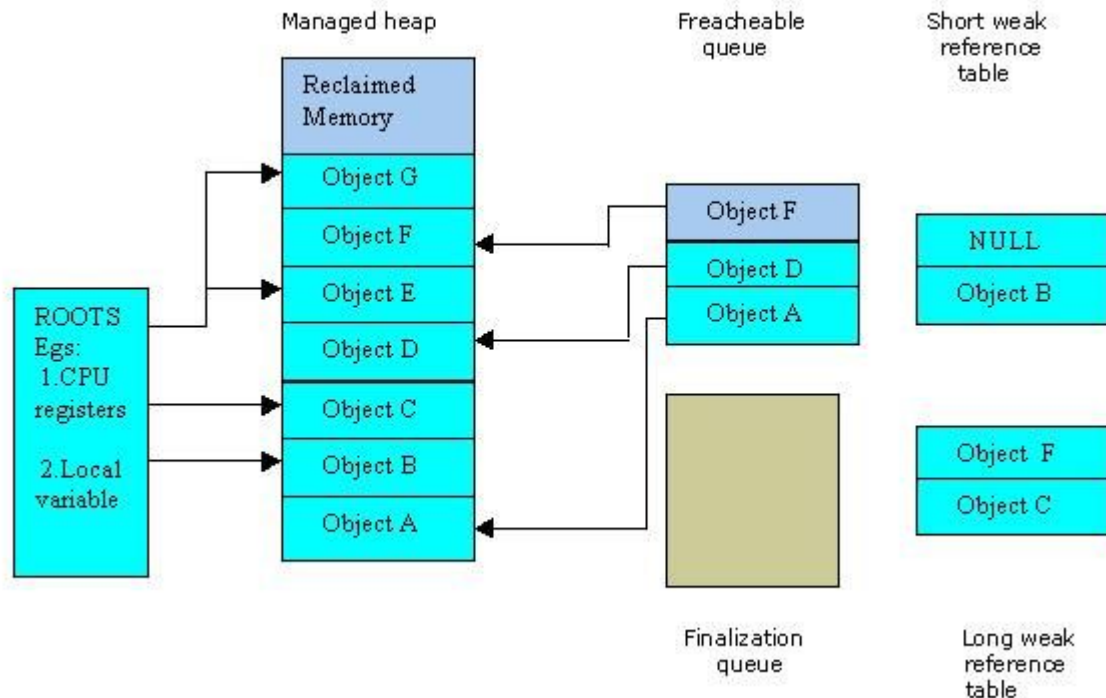
Где хранится WeakReference?

Figure 1 Scenario before the GC runs



Где хранится WeakReference?

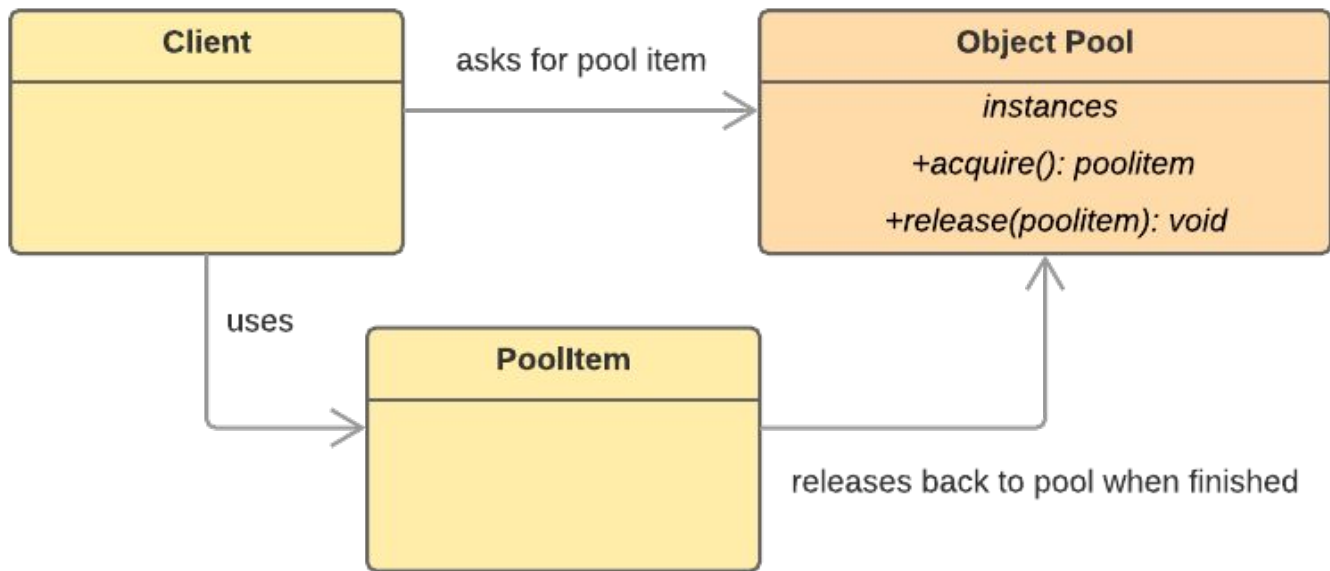
Figure 2 Scenario after the GC runs



Паттерн "Пул объектов"

Пул объектов

Паттерн "пул объектов" заключается в создании ограниченного пула предсозданных объектов, которые многократно переиспользуются вместо регулярного создания и уничтожения при каждом обращении.

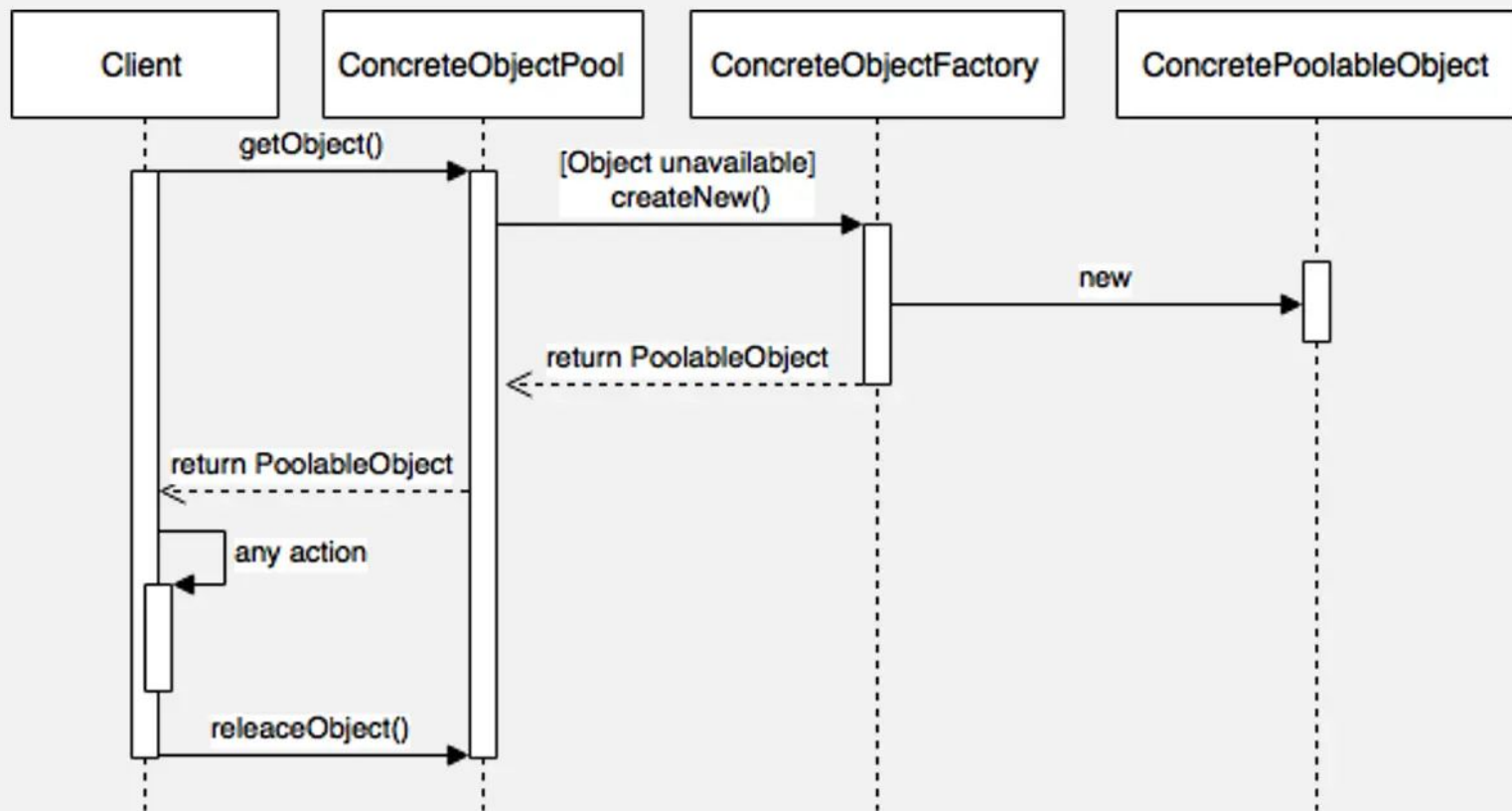


Пул объектов

Классическая схема выглядит так:

1. Инициализация пула — создание фиксированного (или динамически расширяемого) списка/стека объектов требуемого типа.
2. Аренда объекта (Get/Acquire) — клиентский код получает из пула готовый к использованию объект. Если объектов нет, может быть выброшено исключение, произведено создание нового (переполнение), либо процесс ожидает освобождения.
3. Возврат объекта (Return/Release) — после завершения работы объект помещается обратно, и его состояние может быть сброшено (если нужно).

Object Pool pattern – Diagram of sequence



Пул объектов

Подходит для решения ряда проблем:

- сокращение нагрузки на GC,
- оптимизация затрат на выделение и освобождение памяти,
- утилизация "тяжёлых" ресурсов (например, файловых дескрипторов или буферов).

Сценарии применения

- **Работа с внешними ресурсами:** объектов, обёртывающих сетевые соединения (TcpClient, SqlConnection), файловые потоки, сокеты, соединения с очередями/брокерами сообщений.
- **Игровые движки:** обработка частых появлений и исчезновений однотипных элементов (пули, враги, препятствия) без постоянного создания/удаления объектов.
- **Парсеры и сериализаторы:** классы-обработчики, StringBuilder, большие массивы данных.
- **Библиотеки кеширования и работы с буферами:** массивы байтов, блоки памяти, объекты для буферизации данных.
- **Обработка запросов в Web API:** обработчики, middleware-компоненты, объекты для сериализации/десериализации JSON/XML.
- **Высоконагруженные серверные сервисы:** работа с потоками, задачами, обработчиками событий.



Рекомендации

- **Определяйте оптимальный размер пула:** Избыточно большой пул приводит к простоям и ненужному расходу памяти, малый — к задержкам в ожидании или переполнению пула.
- **Используйте политики очистки/сброса состояния объектов:** После возврата объекта его состояние должно быть очищено.. Это исключает ошибки и утечки состояния между клиентами.
- **Обеспечивайте потокобезопасность:** Используйте потокобезопасные коллекции (`ConcurrentBag<T>`, `ConcurrentQueue<T>`), либо стандартные механизмы синхронизации (`lock/Monitor/Mutex`), если пул используется разными потоками.
- **Соблюдайте принцип возврата объектов:** Всегда возвращайте объекты в пул после использования (желательно с помощью конструкции `using/IDisposable`).
- **Избегайте пулов для легковесных объектов**



ArrayPool<T>

ArrayPool<T>

Почему это важно?

- Частое создание массивов → нагрузка на GC
- Высокие аллокации → паузы, снижение производительности
- Нужен механизм повторного использования

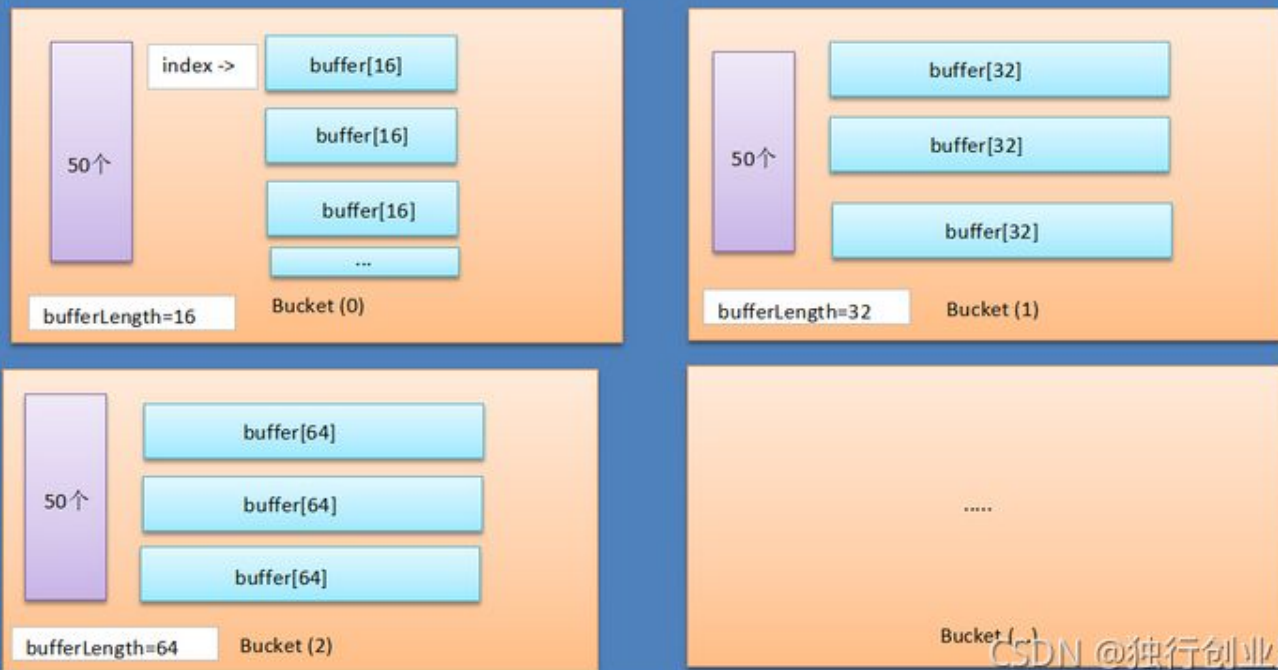
ArrayPool — что это?

- Пул массивов (`T[]`) для повторного использования
- Входит в `System.Buffers`
- Позволяет арендовать и возвращать массивы без лишних аллокаций
- Его исходный код можно найти [здесь](#). **ArrayPool** – это абстрактный класс с двумя реализациями: [SharedArrayPool](#) и [ConfigurableArrayPool](#).



ArrayPool<T>

ArrayPool



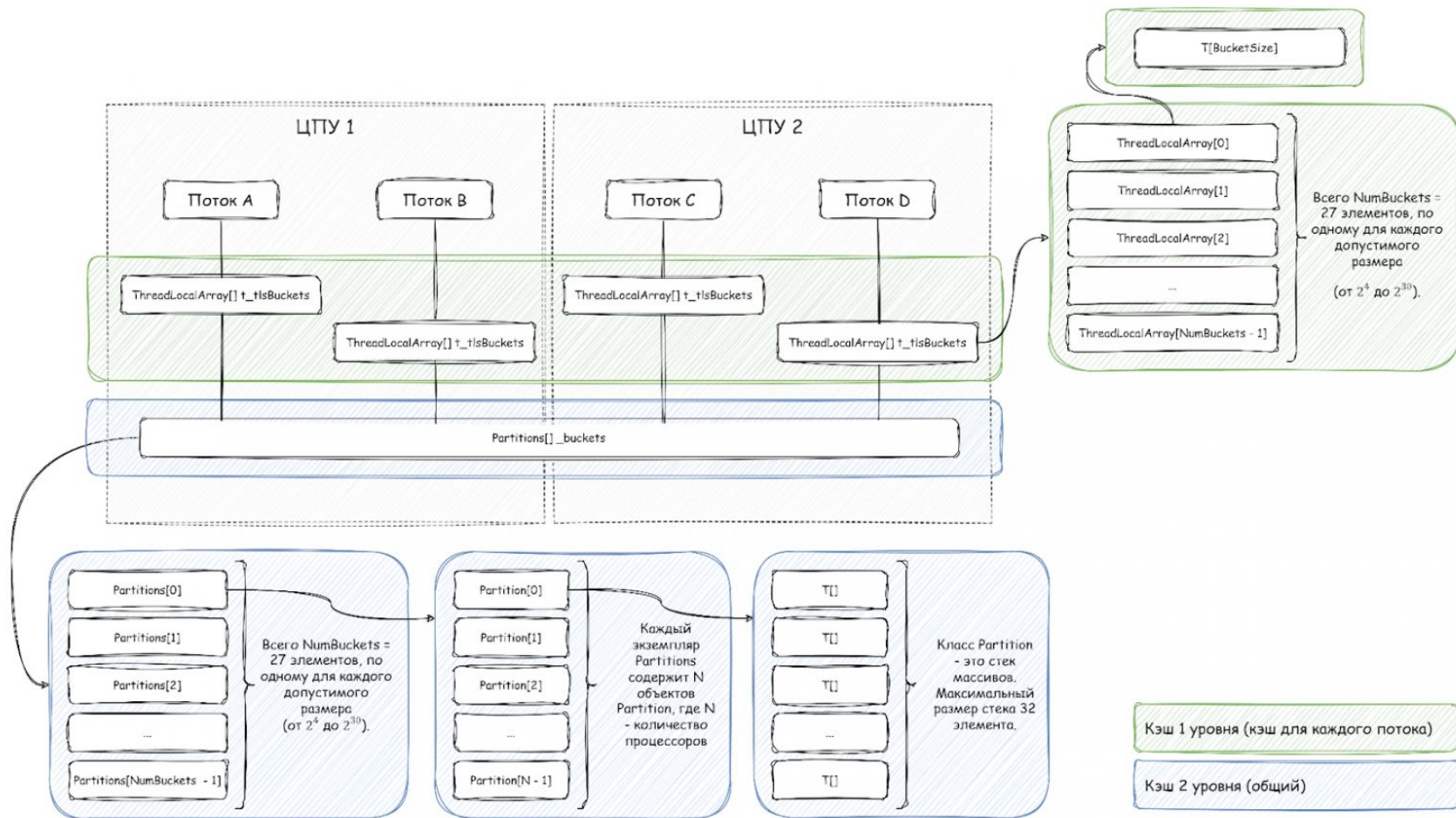
Использование

```
var pool = ArrayPool<int>.Shared;  
var buffer = pool.Rent(10);  
try  
{  
    // do some work with array  
}  
finally  
{  
    pool.Return(buffer, clear: true);  
}
```

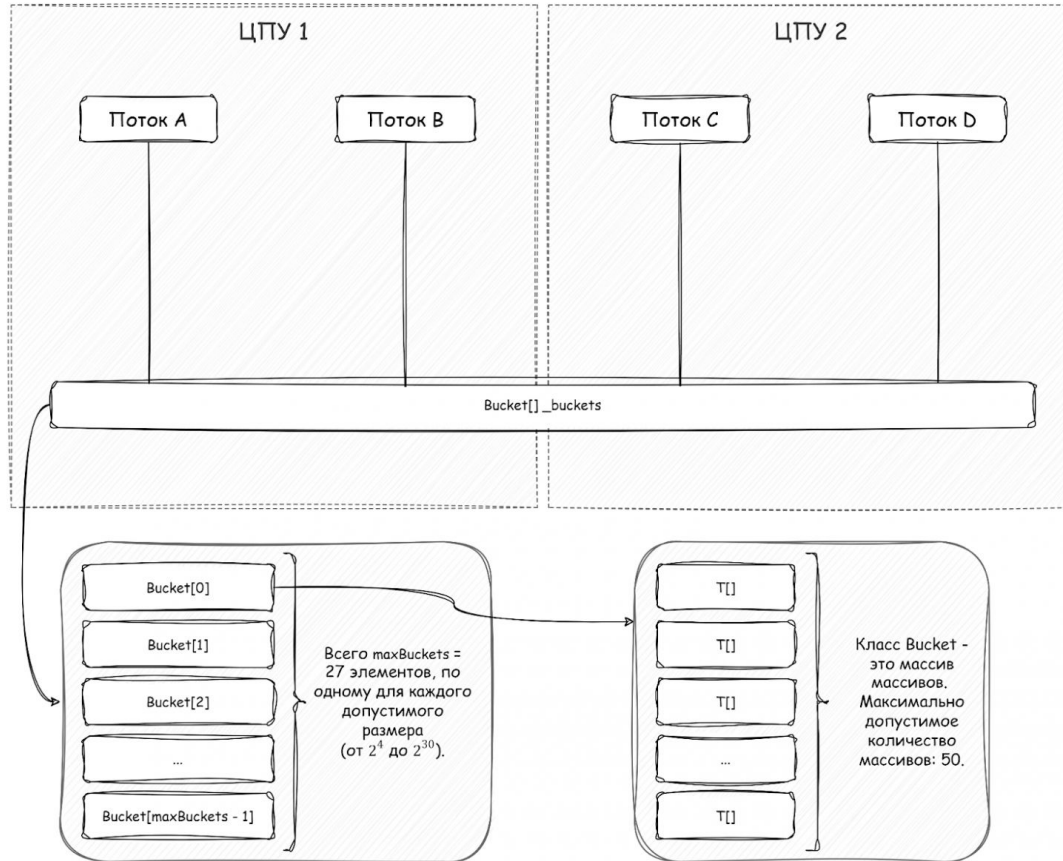
```
var pool = ArrayPool<int>.Create(maxArrayLength: 1000, maxArraysPerBucket: 20);
```



SharedArrayPool<T> под капотом



ConfigurableArrayPool<T> под капотом



ArrayPool<T>

Преимущества

- ✓ Снижение аллокаций
- ✓ Уменьшение давления на GC
- ✓ Повышение производительности
- ✓ Простота использования

Ограничения

- ⚠ Массив может содержать "грязные" данные
- ⚠ Нет автоматической очистки
- ⚠ Ответственность за возврат — на разработчике
- ⚠ Не подходит для сложного управления временем жизни

Применение

ASP.NET Core

В стандартных библиотеках и middleware ASP.NET Core активно используется ArrayPool для буферизации данных, обработки http-запросов/ответов, сериализации/десериализации JSON, парсинга сетевых заголовков. Это один из факторов высокой производительности современных .NET web API

Сетевые библиотеки: Kestrel и gRPC

Во внутренностях сетевого стека Kestrel сервер и в реализации gRPC используется ArrayPool для автоматического управления буферами ввода/вывода, что даёт возможность обрабатывать десятки тысяч запросов в секунду без существенного прироста использования памяти.

Парсеры

Популярные парсеры командных протоколов, реализация механизма buffering в SignalR, image-processing библиотеки, файловые сериализаторы — все эти решения за последние годы массово перешли на использование пулов массивов и показали значительный прирост производительности (до x3) при стабильной нагрузке.

DataScience и анализ биг-дата

Сценарии обработки больших массивов данных — агрегация логов, real-time analytics — используют ArrayPool для буферизации стримов и ускоряют интерпретацию результатов, а также сокращают время задержек при массовых чтениях/записях.



MemoryPool<T>

MemoryPool<T>

Почему обычные массивы не всегда подходят?

- Частые аллокации → нагрузка на GC
- Нет контроля над временем жизни
- Риск утечек и фрагментации
- Нужна безопасная и производительная альтернатива

Что такое MemoryPool?

- Абстракция для аренды буферов памяти
- Возвращает **IMemoryOwner** с **Memory**
- Позволяет безопасно управлять временем жизни
- Может использовать управляемую или неуправляемую память



IMemoryOwner

```
Memory<int> pooledMemory = new Memory<int>(ArrayPool<int>.Shared.Rent(128));  
await Consume(pooledMemory);
```

```
using (IMemoryOwner<int> owner = MemoryPool<int>.Shared.Rent(128))  
{  
    Memory<int> memory = owner.Memory;  
    ConsumeMemory(span);  
    ConsumeSpan(memory.Span);  
}
```



```
public class Worker : IDisposable
{
    private readonly IMemoryOwner<byte> memoryOwner;

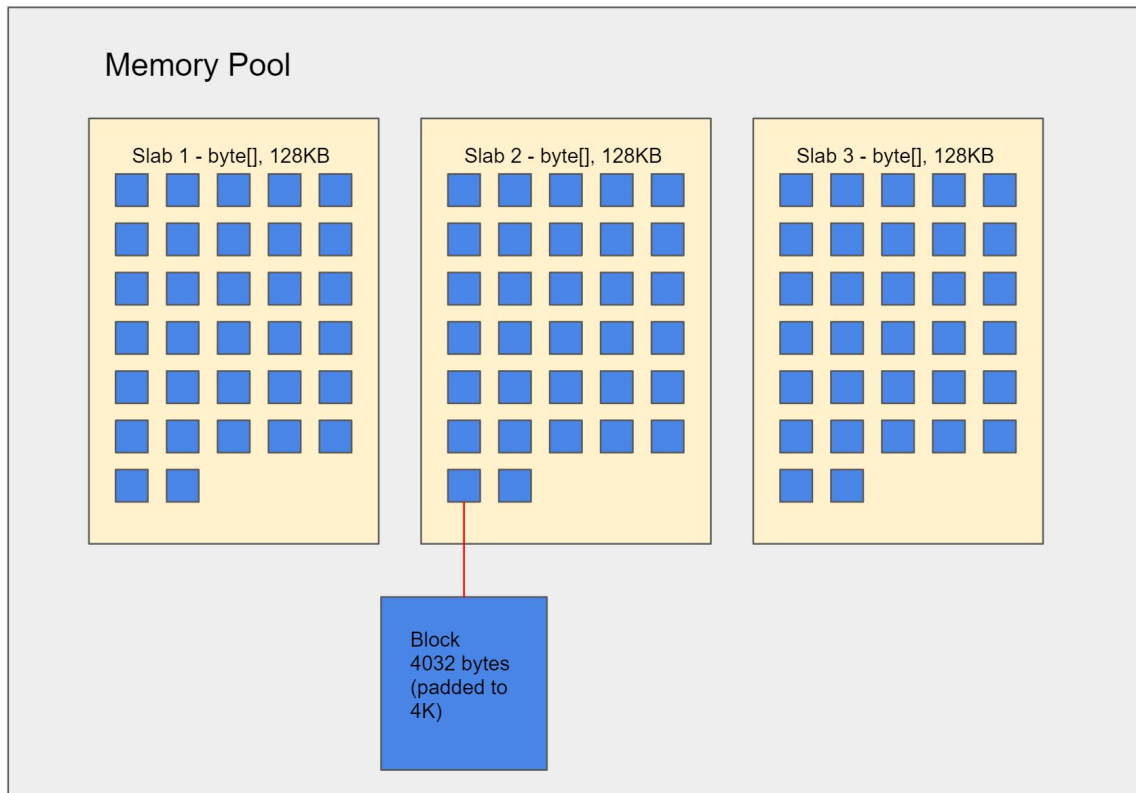
    public Worker(IMemoryOwner<byte> memoryOwner)
    {
        this.memoryOwner = memoryOwner;
    }

    public UseMemory()
    {
        ConsumeMemory(memoryOwner.Memory);
        ConsumeSpan(memoryOwner.Memory.Span);
    }

    public void Dispose()
    {
        this.memoryOwner?.Dispose();
    }
}
```



MemoryPool<T>



Использование

```
const int bufferSize = 4096;

// Использование пула
using (IMemoryOwner<byte> owner = MemoryPool<byte>.Shared.Rent(bufferSize))
{
    Memory<byte> memory = owner.Memory.Slice(0, bufferSize);
    // Теперь memory можно передавать в асинхронные API, пайплайны и т.п.
    // Заполним память данными:
    var span = memory.Span;
    for (int i = 0; i < bufferSize; i++)
        span[i] = (byte)i;

    // После выхода из using память будет возвращена в пул
}

// Здесь блок памяти уже освобождён
```



MemoryPool<T>

Преимущества

- ✓ Безопасное управление временем жизни
- ✓ Поддержка **Span** и **Memory**
- ✓ Возможность использовать неуправляемую память
- ✓ Высокая производительность в потоковых сценариях

Ограничения

- ⚠ Не подходит для долгоживущих буферов
- ⚠ Требуется вызов `Dispose()`
- ⚠ Не предоставляет прямого доступа к массиву (`T[]`)
- ⚠ Не всегда эффективен при очень мелких буферах

Критерий	MemoryPool	ArrayPool
Тип возвращаемого объекта	<code>IMemoryOwner<T></code> с <code>Memory<T></code>	Массивы <code>T[]</code>
Управление ресурсами	Через <code>Dispose()</code> у владельца	Через вызов <code>Return()</code>
Использование с неуправляемой памятью	Да (через кастомные реализации)	Нет, только управляемые массивы
Безопасное владение	Да — после <code>Dispose()</code> доступ невозможен	Нет — доступ к массиву остаётся
Интеграция с <code>Memory/Span</code>	Нативная	Требуется дополнительная обёртка
Сценарии применения	Современные API, пайплайны, асинхронный I/O	Простая работа с массивами
Поддержка <code>IDisposable</code>	Да	Нет



Применение

Высоконагруженные серверные приложения

В приложениях с большими объёмами входящих/выходящих данных (например, веб-серверы, сетевые прокси, системы потоковой обработки данных) позволяет минимизировать количество аллокаций через переиспользование буферов.

Взаимодействие с System.IO.Pipelines

Современный стек ввода-вывода .NET ([System.IO.Pipelines](#)) почти во всех местах использует MemoryPool для выделения временных буферов для чтения/записи данных

ASP.NET Core

В ASP.NET Core для обработки HTTP-запросов и тел больших файлов используется MemoryPool для минимизации расходов на аллокации буферов. Это позволяет повысить производительность серверных приложений, снизить давление на GC и предсказуемо управлять количеством одновременно обрабатываемых запросов (например, внутри Kestrel).

Пакетная и стриминговая обработка данных.

Интеграция с библиотеками сторонних производителей

Многие современные .NET-библиотеки (например, современные реализации сериализации, базы данных, сетевые стеки) внедряют MemoryPool для передачи буферов между слоями и компонентами с семантикой аренды/освобождения.





LIVE



Правила использования

Типовые ошибки

- **"Забыл вернуть"** : Ошибка, при которой буфер не возвращается в пул после обработки, приводит к истощению пула и росту аллокаций вне пула.
- **Избыточная очистка буфера**: Включение `clearArray` на каждом возврате буфера приводит к резкому росту времени обработки, особенно при большом количестве операций.
- **Передача буфера через границы задач** : Если буфер попал "наружу" — например, был сохранён в список, был передан по событию или в `ThreadStatic`-поле — управлять его временем жизни становится невозможно; это прямой путь к утечкам.

Общие рекомендации

- **Гарантируйте возврат ресурса:** Всегда используйте конструкции try/finally (ArrayPool) или using (MemoryPool/IMemoryOwner).
- **Не храните возвращённые ресурсы:** После возврата массива/блока в пул не оставляйте где-либо на них ссылки и не используйте повторно без новой аренды.
- **Минимизируйте время аренды:** Чем меньше времени массив или блок занят у пула, тем выше общая производительность и меньше риск блокировок.
- **Используйте очистку, если это критично:** При возврате массива в ArrayPool используйте параметр clearArray, если массив содержит чувствительные или приватные данные.
- **Проверяйте границы массива:** Арендруемый массив может быть больше запрошенного размера. Используйте только N элементов, где N — изначально требуемый размер.

Шаблон кода для ArrayPool

```
var pool = ArrayPool<byte>.Shared;  
byte[] buffer = pool.Rent(size);  
try  
{  
    // Использовать только buffer[0..size]  
    SomeOperation(buffer, size);  
}  
finally  
{  
    pool.Return(buffer, clearArray: false);  
}
```



Шаблон кода для MemoryPool

```
using (IMemoryOwner<byte> owner = MemoryPool<byte>.Shared.Rent(size))
{
    var span = owner.Memory.Slice(0, size).Span;
    SomeOperation(span);
}
// Гарантированный вызов Dispose
```



Вопросы



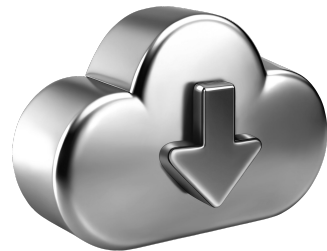
если есть вопросы



если вопросов нет

Список материалов для изучения

1. Konrad Kokosa "Pro .NET Memory Management"
2. [Пулы объектов в C#: примеры, устройство и производительность / Habr](#)
3. [ArrayPool<T>: подводные камни / Habr](#)
4. [ArrayPools to minimize allocations - Tips and tricks](#)



**Подведём
итоги занятия**

Цели вебинара

К концу занятия вы сможете

1. Применять паттерн "пул объектов"

2. Использовать `ArrayPool<T>`

3. Использовать `MemoryPool<T>`

Смысл

Для чего вам это уметь

Проектировать системы, устойчивые к пиковым нагрузкам за счет пулинга буферов



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️